



南開大學
Nankai University

人工智能技术实验 实验报告

实验名称： 无人机编队协同职能
任务规划虚拟仿真实验

姓 名： 殷家兴
学 号： 2313666
专 业： 自动化

人工智能学院

2025 年 12 月

一 问题简述

“无人机编队协同智能任务规划虚拟仿真实验”内容包括无人机的自主起降、路径规划、遍历覆盖、编队协同。实验采用离线编程方式实现数据驱动的虚拟场景实验及交互，可灵活设计智能决策、控制、规划及视觉检测、跟踪、重建算法，并进行效果评价。本实验能够真实模拟军事战场、生态保护、国土资源管理等领域多无人机协同任务，训练学生探索智能控制、规划与任务决策方法的能力。

二 实验目的

1. 了解无人机编队协同智能任务规划的重要性、必要性、先进性等；
2. 掌握无人机控制、规划、决策的基本原理及方法；
3. 掌握无人机自主起降、路径规划、遍历覆盖、编队协同的算法及控制策略；
4. 支持学生自主设定任务目标，进行离线编程驱动虚拟仿真场景以验证任务完成度、整体性能并与给定方法进行比对评价；
5. 培养学生利用虚实结合的技术手段解决实际问题的能力。

三 实验内容

实验包括自主起降、路径规划、遍历覆盖、编队协同四个子任务，每个子任务的实验内容如下：

1. 通过基于位置/图像的视觉伺服控制方法实现无人机自主起降功能；
2. 通过基于贝塞尔平滑的多端连续加速度级轨迹规划方法实现多途径点的路径规划；
3. 通过遗传算法实现视觉遍历覆盖；
4. 通过蚁群算法实现多无人机编队协同智能任务规划算法。

四 实验步骤

该实验共计 6 个实验学时，15 个交互性操作步骤。实验流程为：进入实验 → 理论学习与知识测试 → 交互式编程环境测试 → 虚拟摄像机设置 → 无人机加载与初始化 → 实验任务（自主起降、路径规划、遍历覆盖、编队协同）→ 生成实验结论 → 实验结束并发布成绩，下图为实验项目操作流程图。

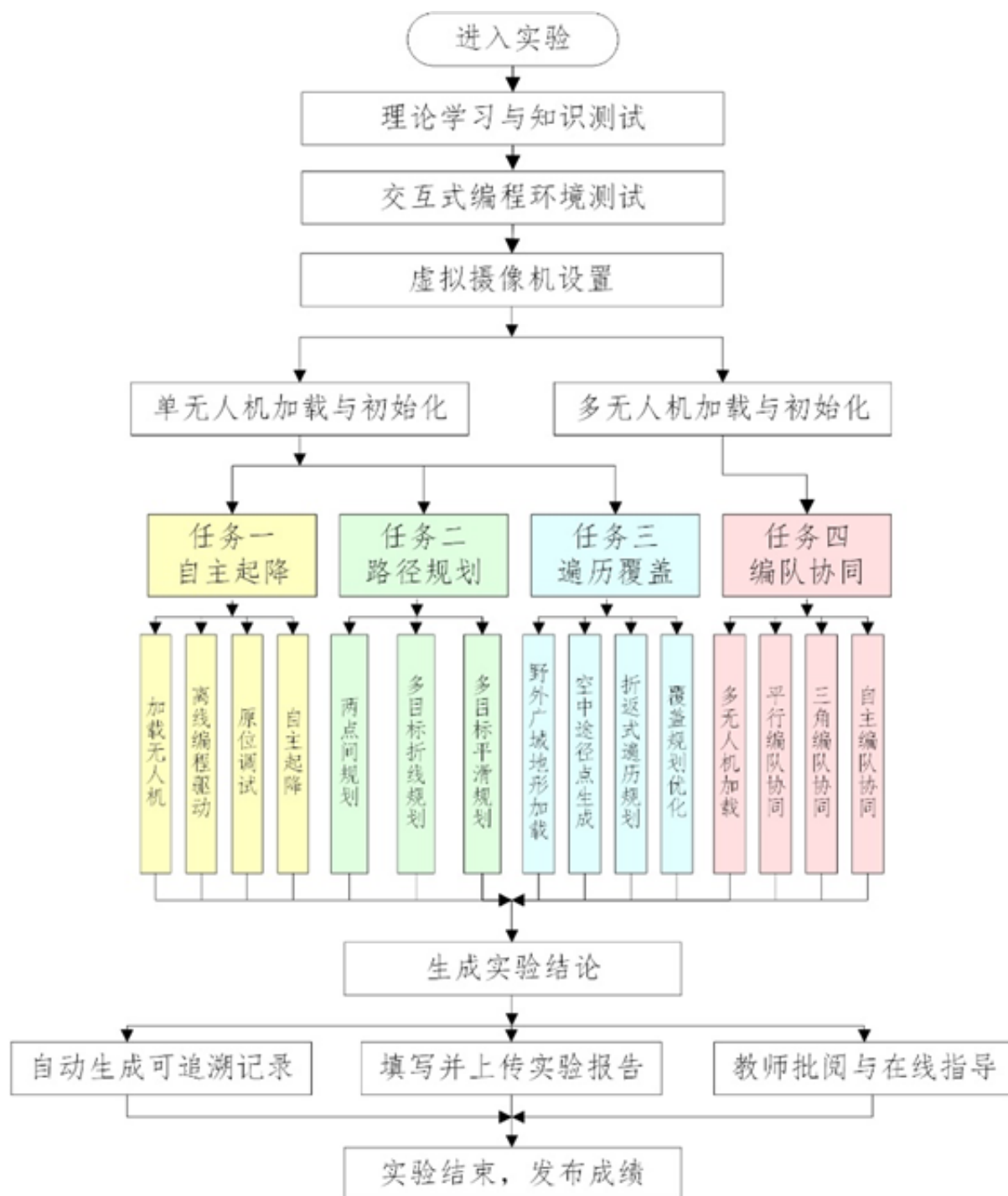


图 1: 实验操作流程

五 自主设计实验

(针对一个实验任务或者多个实验任务对实验平台的已有算法进行分析或者改进)

下面对“任务三：遍历覆盖”进行分析，着重分析使用遗传算法的任务 3-1 中。

本任务核心需求是对目标区域进行完整、高效且高质量的空中观测，通过模拟无人机在已知三维山体地形上空，自动生成满足需求的空中观测点序列，并规划一条能够遍

历所有观测点的飞行轨迹。

5.1 山体表面法向量估计

给定山体网格数据 (X_{ij}, Y_{ij}, Z_{ij}) ，对于任意地面点 (x, y) ，通过最近邻搜索找到对应的格点 $C = (x_0, y_0, z_0)$ 。为估计该点处的单位法向量 n ，采用八邻域叉积平均法：

- 构造 8 个指向相邻格点的向量 $v_k = P_k - C$;
- 对每对相邻向量计算叉积: $n_k = v_k \times v_{k+1}$;
- 归一化后取平均:

$$\mathbf{n} = \frac{1}{8} \sum_{k=1}^8 \frac{\mathbf{n}_k}{|\mathbf{n}_k|} \quad (1)$$

该方法能有效逼近曲面局部法向，避免因网格不规则导致的误差。

该部分内容对应代码如下：

```
1 def GetNormalVector(self, CurrCx, CurrCy):
2     """
3     计算山体表面离所给点(CurrCx, CurrCy)最近的格点并求解该格点处的单位法向量
4     """
5     eDis2 = (self.X - CurrCx)**2 + (self.Y - CurrCy)**2
6     Idx = np.where(eDis2.min() == eDis2)
7     row = Idx[0][0]
8     col = Idx[1][0]
9     CPoint=np.asarray([ \
10         self.X[row, col],
11         self.Y[row, col],
12         self.Z[row, col],
13     ]).reshape(3,1)
14     CIdx = [row, col]
15     # 八邻域相对位置(正右开始逆时针转动[dRow, dCol])
16     eightDirect = [ \
17         [0, 1],
18         [-1, 1],
19         [-1, 0],
20         [-1, -1],
21         [0, -1],
22         [1, -1],
23         [1, 0],
24         [1, 1]
25     ]
26     # 指向八个方向的向量
27     eightCDVList = []
```

```

28     for i in range(0, 8):
29         CDV = np.asarray([ \
30             self.X[row + eightDirect[i][0], col +
eightDirect[i][1]],
31             self.Y[row + eightDirect[i][0], col +
eightDirect[i][1]],
32             self.Z[row + eightDirect[i][0], col +
eightDirect[i][1]],
33             ]).reshape(3, 1) - CPoint
34         eightCDVList.append(CDV)
35     eightCDVList.append(eightCDVList[0])
36     # 相邻方向向量叉乘得到的八个近似法向量
37     eightCNVList = []
38     for i in range(0, 8):
39         CNV = np.cross(eightCDVList[i], eightCDVList[i + 1], axis=0)
40         eightCNVList.append(CNV / np.linalg.norm(CNV))
41     eightCNV = np.hstack(eightCNVList)
42     # 八个近似法向量取平均即为正式法向量
43     CNV = eightCNV.mean(axis=1).reshape(3, 1)
44     CNV = CNV / np.linalg.norm(CNV)
45     return CNV, CPoint, CIdx

```

5.2 相机视点生成

设定相机视野参数：

- 视野宽度 cW 、高度 cH ；
- 横向重合率 cWr 、纵向重合率 cHr ；
- 飞行高度（沿法向距离） dis 。

根据覆盖约束，计算所需视点数量：

$$N_x = 1 + \left\lfloor \frac{W + cW \cdot cWr}{cW(1 - cWr)} \right\rfloor \quad N_y = 1 + \left\lfloor \frac{H + cH \cdot cHr}{cH(1 - cHr)} \right\rfloor \quad (2)$$

采用“之字形”扫描策略，在每个地面中心点 C_i 处沿法向偏移 dis 得到空中视点：

$$V_i = C_i + dis \cdot \mathbf{n}_i \quad (3)$$

该部分内容对应代码如下：

```

1 def GetVpoints(self, cW=512, cH=288, cWr=0.3, cHr=0.2, dis=466):
2     """
3     根据等距正视约束计算空中 vPoints 相机视野宽度高度依次为 cW、cH 相机视野横向
    重合率、纵向重合率依次为 cWr、cHr 等距所指的距离为 dis

```

```

4      """
5      ## 定义左上角与右下角
6      Xstart=np.asarray([ \
7          self.X[0,0],
8          self.Y[0,0],
9          self.Z[0,0]
10         ]).reshape(3,1)
11      Xend=np.asarray([ \
12          self.X[-1,-1],
13          self.Y[-1,-1],
14          self.Z[-1,-1]
15         ]).reshape(3,1)
16      ## 求相机中心点位置序列
17      W = Xend[0] - Xstart[0]
18      H = Xend[1] - Xstart[1]
19      # 恰好有  $ciN*cW-(ciN+1)*cW*cWr$   $W$ 
20      ciN = 1 + int((W + cW * cWr) / (cW - cW * cWr))
21      # 恰好有  $cjN*cH-(cjN+1)*cH*cHr$   $H$ 
22      cjN = 1 + int((H + cH * cHr) / (cH - cH * cHr))
23      # 遍历网格进行寻找
24      CNVList = []
25      CPointsList = []
26      CIdxsList = []
27      VPointsList = []
28      dLabel = 1
29      for ci in range(1, ciN + 1):
30          CurrCx = Xstart[0] + ci * (cW - cW * cWr) - cW / 2
31          for cj in range(1 * (dLabel == 1) + cjN * (dLabel == -1),
32              (cjN + 1) * (dLabel == 1), dLabel):
33              CurrCy = Xstart[1] + cj * (cH - cH * cHr) - cH / 2
34              CNV, CPoint, CIdx = self.GetNormalVector(CurrCx, CurrCy)
35              VPoint = CPoint + CNV * dis
36              CNVList.append(CNV)
37              CPointsList.append(CPoint)
38              CIdxsList.append(CIdx)
39              VPointsList.append(VPoint)
40          dLabel = -dLabel #实现之字形遍历
41      self.CNVs = np.hstack(CNVList)
42      self.CPoints = np.hstack(CPointsList)
43      self.CIdxsList = CIdxsList
44      self.VPoints = np.hstack(VPointsList)
45      self.KeyX = np.hstack([Xstart * 0] + VPointsList + [Xend * 0])
46      self.KeyV = self.KeyX * 0
47      self.KeyA = self.KeyX * 0

```

5.3 TSP 建模与遗传算法求解

将所有视点（含起点 $S = (0, 0, 0)$ 与终点 $E = (0, 0, 0)$ ）视为图中节点，边权为欧氏距离。目标是最小化总飞行距离：

$$\min \sum_{i=1}^N \|P_{\pi(i+1)} - P_{\pi(i)}\| \quad (4)$$

其中 π 为访问顺序排序，且 $\pi(1) = S$ ， $\pi(N) = E$ 。

由于 TSP 是 NP-hard 问题，采用遗传算法（GA）近似求解：

- **染色体编码**：路径索引序列；
- **适应度函数**：总路径长的倒数；
- **选择、交叉、变异**：标准操作（如顺序交叉 OX）；
- **终止条件**：最大代数（如 2000 代）或收敛。

该部分内容对应代码如下：

```
1 def SolveDisTSP(self):
2     """
3     求解基于距离的 TSP 问题，得到比较优秀的遍历顺序。由于遗传算法求解比较
4     耗时，应该离线进行，故此处采用 os.system() 函数终端调用命令。如果已有离线
5     进行的结果则不再调用，从而加快速度。如此一来，对于特定山体，第一次调用会
6     比较费时，之后就不费时了。
7     """
8     try:
9         pyVI = np.loadtxt('./Workspace/pyVI-GenTotal_2000.res')
10        if pyVI.shape[0] - 1 == self.KeyX.shape[1] \
11            and pyVI[1:].max() == self.KeyX.shape[1]-1 \
12            and pyVI[1:].min() == 0 \
13            and pyVI[1] == 0 \
14            and pyVI[-1] == self.KeyX.shape[1]-1:
15            self.pyVI = pyVI
16            return
17        except BaseException:
18            pass
19        ## 生成 TSPmain.py 所需的文件
20        DstDir = './Workspace'
21        if not os.path.exists(DstDir):
22            os.makedirs(DstDir)
23        fid = open('./keyXVA.txt', 'wb')
24        np.savetxt(fid,
25                    np.vstack([self.KeyX, self.KeyV, self.KeyA]),
```

```

23         fmt='%.17g',
24         delimiter=' ',
25         newline='\n',
26         header='',
27         footer='',
28         comments='# ')
29     fid.close()
30     ## 终端调用 TSPmain.py 求解
31     cmdStr = 'python3 TSPmain.py -res ./Workspace/pyVI-GenTotal_2000.
res -Nind 100 -MGen 2000 -drawType 0'
32     os.system(cmdStr)
33     ## 读取求解结果
34     self.pyVI = np.loadtxt('./Workspace/pyVI-GenTotal_2000.res')

```

5.4 基于遗传算法 TSP 遍历顺序的贝塞尔轨迹规划

为使无人机运动平滑，需保证位置、速度、加速度连续。采用五次多项式参数化贝塞尔曲线。

5.4.1 参数化函数 $m(t)$

设时间 $t \in [0, t_f]$ ，定义标量函数 $m \in [0, 1]$ 满足：

$$\begin{cases} m(0) = 0, & m(t_f) = 1 \\ m'(0) = Dm_0, & m'(t_f) = Dm_1 \\ m''(0) = DDm_0, & m''(t_f) = DDm_1 \end{cases} \quad (5)$$

令 $m(t) = \sum_{k=0}^5 c_k t^k$ ，构建线性方程组 $Ac = b$ 求解系数 c 。

5.4.2 控制点计算

给定起点 X_s 、终点 X_e 及其速度 V_s, V_e 、加速度 A_s, A_e ，五次贝塞尔曲线有六个控制点 $P_0 P_5$ ：

$$\begin{aligned}
P_0 &= X_s \\
P_5 &= X_e \\
P_1 &= P_0 + \frac{V_s}{5Dm_0} \\
P_4 &= P_5 - \frac{V_e}{5Dm_1} \\
P_2 &= \frac{A_s - \frac{V_s}{Dm_0}DDm_0}{20Dm_0^2} - P_0 + 2P_1 \\
P_3 &= \frac{A_e - \frac{V_e}{Dm_1}DDm_1}{20Dm_1^2} - P_5 + 2P_4
\end{aligned} \tag{6}$$

此推导基于贝塞尔曲线导数性质。

5.4.3 轨迹采样

对每段曲线，使用 Berrnstien 基函数计算轨迹点：

$$B(m) = \sum_{k=0}^5 \binom{5}{k} m^k (1-m)^{5-k} P_k \tag{7}$$

其中 $m = m(t)$ 。

贝塞尔轨迹规划部分对应代码如下：

```

1  def SolveBezier(self):
2      """
3      在遗传算法TSP遍历顺序基础上进行贝塞尔规划
4      """
5      ## 初始化贝塞尔求解器类对象
6      Ibs = BezierSolver()
7      ## 导入关键点序列
8      keyX = self.KeyX[:, self.pyVI[1:].astype('int')]
9      keyV = self.KeyV[:, self.pyVI[1:].astype('int')] * 0
10     keyV[:, 1:-1] = keyX[:, 2:] - keyX[:, 1:-1]
11     keyV[:, 1:-1] = keyV[:, 1:-1] / np.linalg.norm(keyV[:, 1:-1], axis=0)
12     keyV[:, 1:-1] = keyV[:, 1:-1] * 17
13     keyA = self.KeyA[:, self.pyVI[1:].astype('int')]
14     ## 求解各段贝塞尔控制点
15     self.PcsMse = Ibs.Solve_B5Special(keyX, keyV, keyA)
16     ## 求解各段轨迹
17     self.Pts = Ibs.GetPt(self.PcsMse)

```

5.5 算法流程

1. 加载山体数据
2. 生成地面中心点与空中观点
3. 构建 TSP 问题并调用遗传算法求解
4. 根据最优顺序提取关键点序列
5. 计算中间点速度方向
6. 分段求解五次贝塞尔控制点
7. 采样生成密集轨迹点
8. 将结果发送至仿真平台执行任务

5.6 其他分任务调用方法对比

表 1: 三种山体覆盖算法处理流程对比

处理阶段	3-2.BFSG 算法	3-3.GACPPSG 算法	3-4.SmoothBF 算法
视点生成		完全相同	
路径规划	固定策略：采用简单的“之字形”扫描模式遍历 vPoints。计算速度快，但路径长度非最优。	智能优化：调用外部遗传算法求解问题 (TSP)，寻找近似最短的全局遍历路径。计算开销大，但路径更优。	智能优化：与 GACPPSG 相同，也使用遗传算法求解 TSP。
轨迹生成	Stop-To-Go：在贝塞尔曲线生成后，强制将中间控制点退化，使无人机在每个 vPoint 处完全停止。轨迹不连续。	Stop-To-Go：与 BFSG 相同，同样采用强制退化策略，牺牲平滑性换取精确停靠。	平滑轨迹：保留了完整的五阶贝塞尔曲线，未进行任何退化操作。生成的轨迹在位置、速度、加速度上均保持连续，飞行过程流畅。

六 实验结果

(国家虚拟仿真实验教学课程共享平台上的成绩截图，90 分以上不扣分)

国家虚拟仿真实验教学课程共享平台上的成绩截图如下：



The screenshot shows a web interface with a table of experimental results. The table has columns for experiment name, student name, student ID, result, score, start time, end time, duration, and actions. One row is visible, showing a score of 100 for a drone mission planning experiment.

实验名称	姓名	学号	实验结果	实验成绩	实验开始时间	实验结束时间	实验用时	操作
无人机编队协同智能任务规划虚拟仿真实验	殷家兴	2313666	完成	100	2025-12-03 16:39:37	2025-12-03 17:33:54	55	步骤详情

1 共 1 条

图 2: 成绩截图

七 分析总结

本次实验基于国家虚拟仿真实验教学课程共享平台和南开大学虚拟仿真实验教学项目共享平台展开，围绕无人机控制进行学习、实验。

通过本次实验，我对无人机控制方法有了一定的学习和了解，并且对以遗传算法为代表的多种控制算法有了一定的了解。遗传算法通过求解 TSP 获得近似最短的访问路径，并结合完整的五次贝塞尔曲线生成位置、速度、加速度均连续的平滑轨迹，兼顾效率与飞行品质；BFSG 算法采用固定的“之字形”扫描策略，路径未优化、总长度较长，且在每个视点强制停顿，轨迹不连续，但计算速度快、实现简单；GACPPSG 算法虽也使用遗传算法优化路径，却沿用了 Stop-To-Go 的停顿策略，牺牲了平滑性以换取精确悬停；而 SmoothBF 算法则在同样优化路径的基础上，保留完整的贝塞尔插值，实现了最优路径与高阶连续轨迹的统一。